

Tuning the Step: How Wolfe Parameters Shape Optimization Performance

Team: Bazinga Trio

Member: Yukai Wang, Jincheng Ye, Yuwei Zhuo

University Of Michigan | IOE 511

Introduction

The performance of optimization algorithms using Wolfe line search depends critically on the choice of parameters **(c1,c2)**, yet their practical impact is not well understood.

In this study, we investigate the sensitivity of algorithm performance to these parameters and identify choices that perform well on average. We focus on four representative methods—Gradient Descent, Newton, BFGS, and DFP—to examine how parameter selection interacts with different search directions.

Wolfe Line Search Conditions

$$\begin{aligned} f(x_k + \alpha_k d_k) &\leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k \\ f(x_k + \alpha_k d_k)^\top d_k &\geq c_2 \nabla f(x_k)^\top d_k \end{aligned}$$

Average Performance

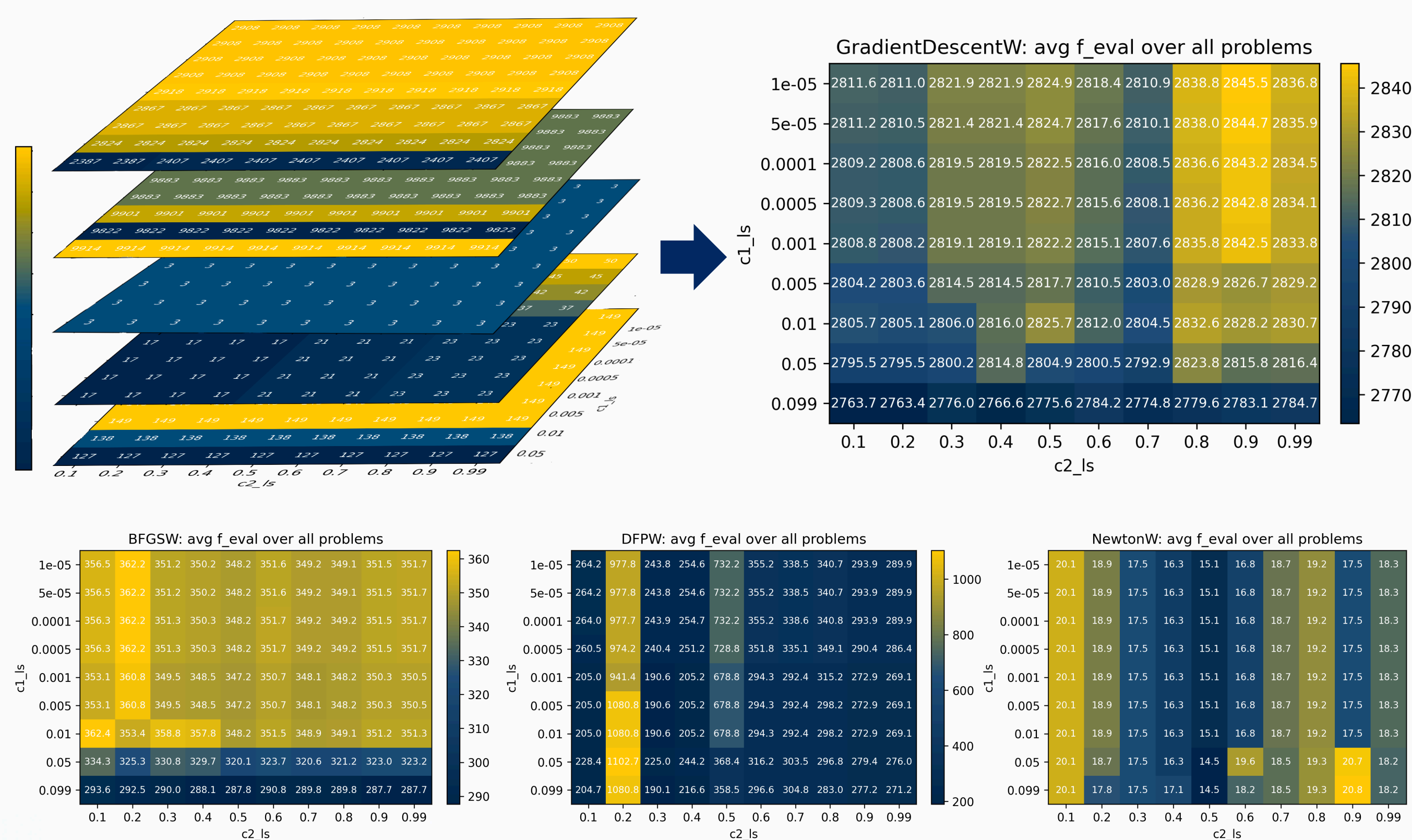


Figure 1: Average performance (iteration amount) of each algorithm among different problems.

Sensitivity Analysis

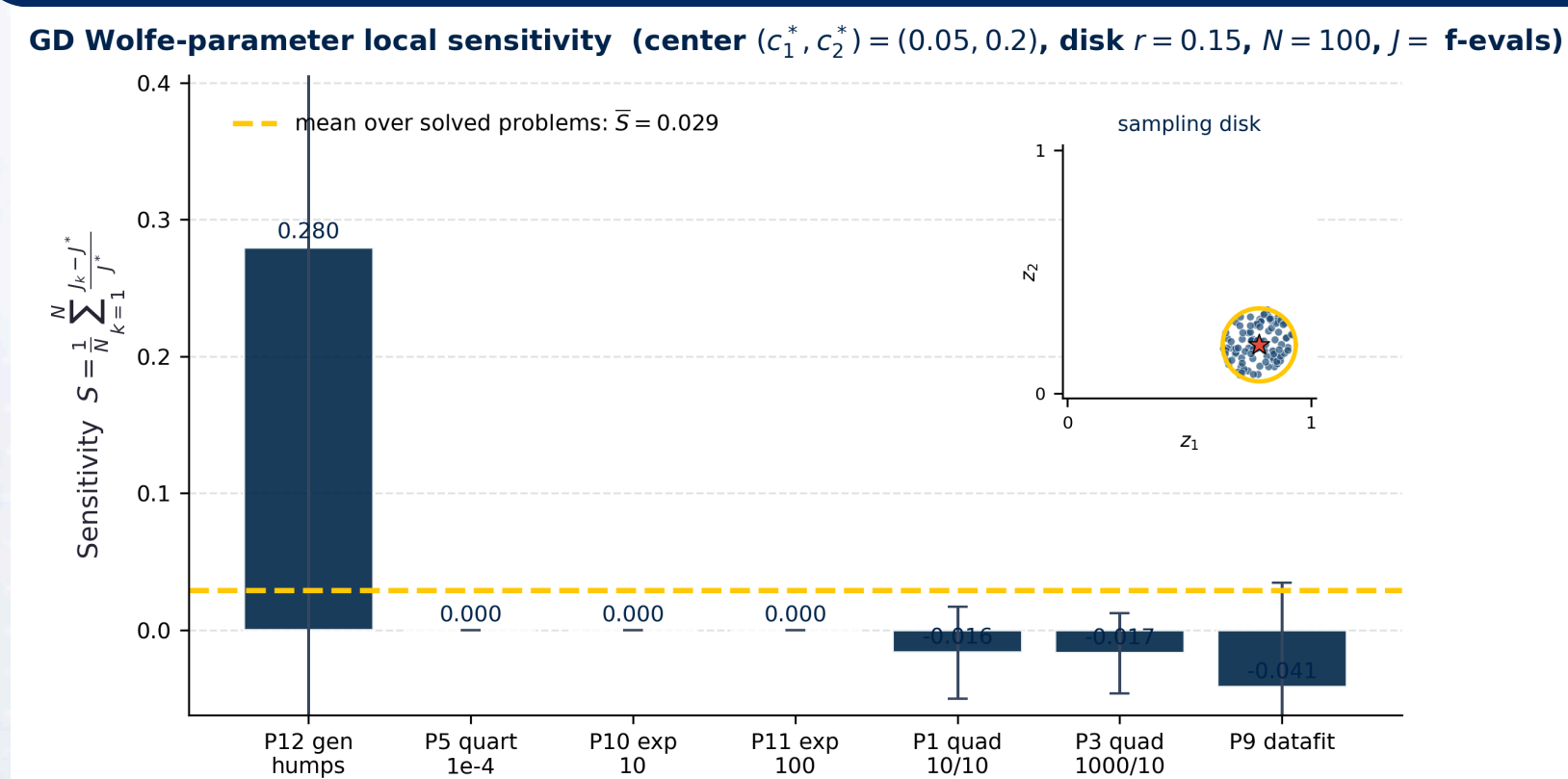


Figure 5: Around $(c1 *, c2 *) = (0.05, 0.20)$, in the normalized parameter space Nr , we uniformly sample $N=100$ points and use $J = f_eval$ as the performance metric.

$$S_p = \frac{1}{N} \sum_{k=1}^N \frac{J_k - J_p^*}{J_p^*} \quad z_1 = \frac{\log_{10}(c_1) - \log_{10}(c_{1,\min})}{\log_{10}(c_{1,\max}) - \log_{10}(c_{1,\min})}, \quad z_2 = \frac{c_2 - c_{2,\min}}{c_{2,\max} - c_{2,\min}}$$

$$N_r = \left\{ (u, v) : (z_1^*(u) - z_1^*)^2 + (z_2(v) - z_2^*)^2 \leq r^2 \right\}$$

J_k is measured at the k -th sample in N_r , and J_p^* is measured at the center.

J_k is measured at the k -th sample in N_r , and J_v^* is measured at the center.

Algorithms

- GD
- Newton
- BFGS
- DFP

Range of Value for (c1,c2)

- | c1 | $0 < c1 < c2 < 1$ | c2 |
|---|-------------------|---|
| <ul style="list-style-type: none"> controls acceptance condition too large $\rightarrow$ overly strict | | <ul style="list-style-type: none"> controls aggressiveness too small $\rightarrow$ overly conservative |
| (1e-5, 0.05) | | (0.1, 0.99) |

Performance Profiles

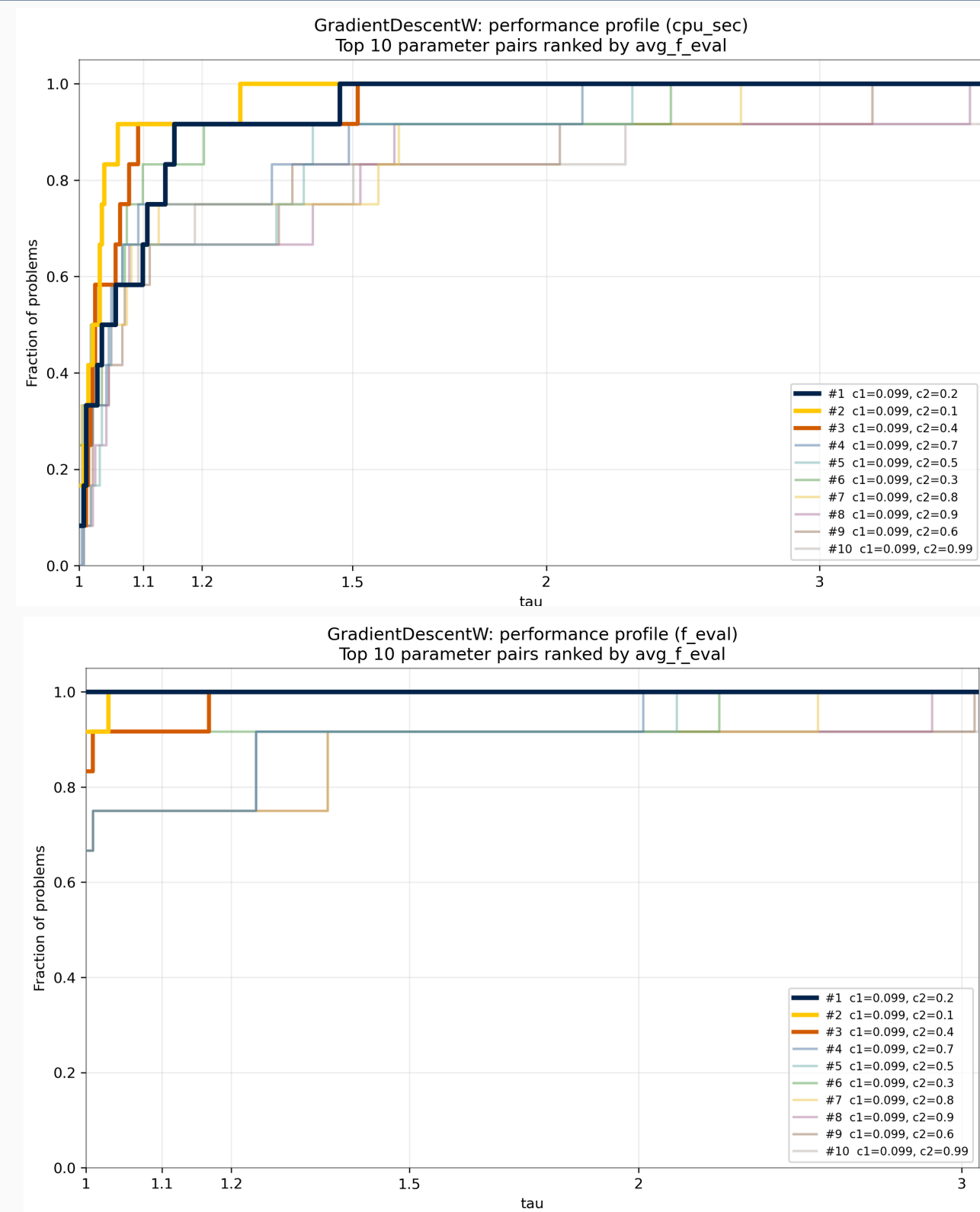


Figure 2: Example: Although **(0.077, 0.2)** for GD method shows slightly better outcome than **(0.077, 0.1)**, its CPU running time is distinctly longer. Overall, we should choose **(0.077, 0.1)**.

Average vs. Performance Profiles

- Average performance summarizes overall efficiency across problems, but it is sensitive to outliers and may favor parameter choices that perform extremely well on a small subset of cases (Fig. 2).
- Performance profiles emphasize robustness by measuring how consistently a method performs near the best across a wide range of problems. **Best Choice**

In our study, we observe that the parameter setting with the best average performance does not always achieve the best performance profile. To identify reliable parameter choices, we adopt a combined criterion:

- First step – Average performance:
select top-performing parameter pairs to filter out inefficient choices
- Second step – Performance profile:
compare shortlisted candidates and select those with most robust performance

Method	(c1,c2)
GD	(0.077,0.7) (0.077,0.1)
Newton	(0.05,0.5)
BFGS	(0.077,0.77) (0.077,0.7)
DFP	(0.001,0.5)

Conclusion

Wolfe parameters matter, but their impact is uneven. To evaluate global robustness, we listed performance profiles of the top **10** parameter pairs, identifying a small **c1** and a large **c2** as the best average choices. Complementing this, our local sensitivity analysis shows that around GD's optimum, a **15%** perturbation in the normalized $\log c_1, c_2$ space changes f-evals by under **5%** on **6/7** convergent problems. The lone outlier is the non-convex genhumps problem. Ultimately, GD is insensitive where it works and sensitive where it struggles—precise tuning is primarily needed for hard, non-convex landscapes.